



THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

OBJECT-ORIENTED DESIGN OBJECTIVES AND PRINCIPLES (EXERCISES AND QA)

COMPSCI 230 OBJECT-ORIENTED SOFTWARE DEVELOPMENT

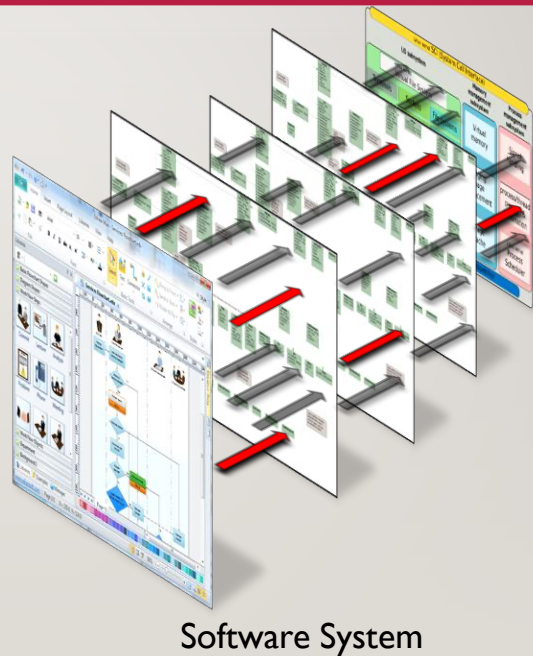
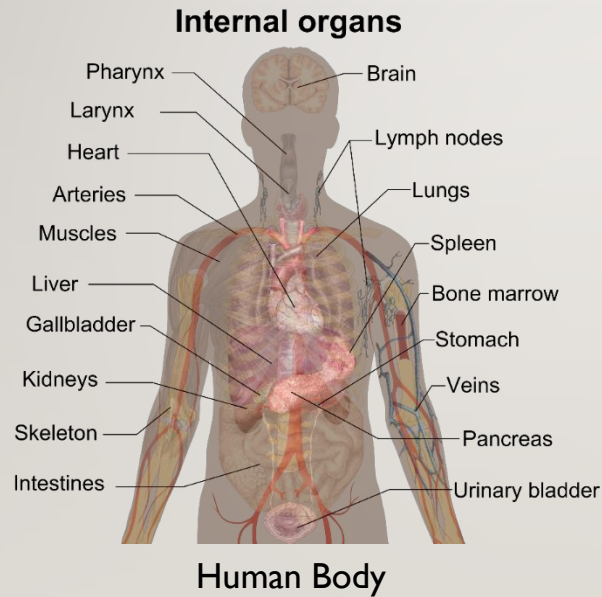
PARAMVIR SINGH

- ✓ Single Responsibility Principle
- ✓ Open Closed Principle
- ✓ Liskov Substitution Principle
- ✓ Interface Segregation Principle
- ✓ Dependency Inversion Principle

AGENDA

- Software design objectives
- SOLID OO design principles

WHY DO SYSTEMS FAIL?



“Undesirable dependencies” fail systems

SELECTED OBJECTIVES FOR SOFTWARE DESIGN

- **Correctness**
 - Software should clearly conform to its specification
- **Efficiency**
 - Refers to processor cycles and memory consumption
 - Software should execute as fast as required and use no more memory than necessary
- **Testability**
 - Designing software for ease of testing
- **Reusability**
 - Any engineering process is concerned with creating useful products, of specified quality, at minimal cost
- **Robustness**
 - Software is robust if it can handle unusual conditions such as corrupt data, user error, programmer error, and environmental conditions
- **Flexibility**
 - Software should be easy to change due to chained dependencies

EXAMPLE TECHNIQUES FOR ACHIEVING TWO OBJECTIVES

- **Robustness**

- *Example Solutions*

- Verify parameter values in methods before using them
 - Enforce design intentions through use of appropriate programming language constructs (e.g. access modifiers, exception handling, interfaces)

- **Flexibility**

- **Rigidity**

- Software is difficult to change due to chained dependencies.

- **Fragility**

- Software breaks at unexpected places when a change is made at a single place.

- *Example Solutions*

- Program to interfaces, not implementations
 - Use inheritance hierarchies for multiple types, rather than a single class with switch statements to discriminate types
 - Encapsulate details that are likely to change so that any change is localised

OO DESIGN PRINCIPLES

- Single Responsibility Principle (SRP)
- Open Closed Principle (OCP)
- Liskov Substitution Principle (LSP)
- Interface Segregation Principle (ISP)
- Dependency Inversion Principle (DIP)



✓ Robert Martin, 2000: *Design Principles and Design Patterns*, published @www.objectmentor.com

A red Swiss Army knife with multiple tools extended, including a saw, a corkscrew, and a pair of pliers. The knife is shown from a side-on perspective, highlighting its various functions. The red handle is prominent, and the silver-colored metal tools are spread out against a white background.



SINGLE RESPONSIBILITY PRINCIPLE

- **“A module (or class) must have one, and only one reason to change”**
- **A module (or class) must have one, and only one responsibility**

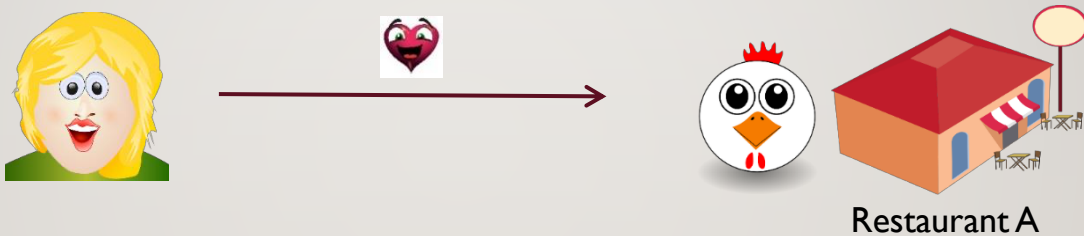


```
class Employee {  
    public Pay calculatePay() {...}  
    public void saveEmployeeInfo() {...}  
    public String generateEmployeeReport() {...}  
}
```

Example

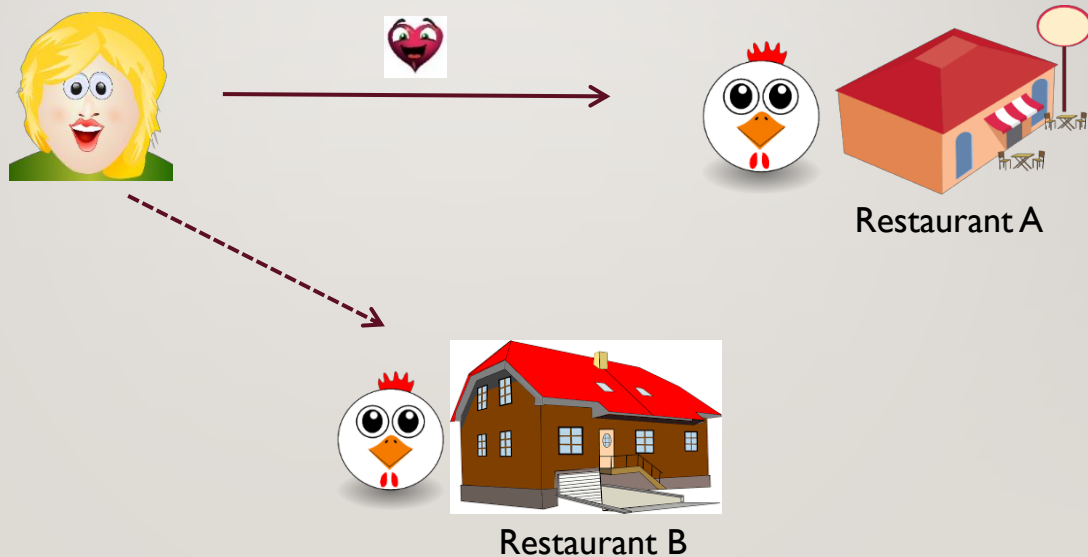
OPEN CLOSED PRINCIPLE

➤ A Real World Example



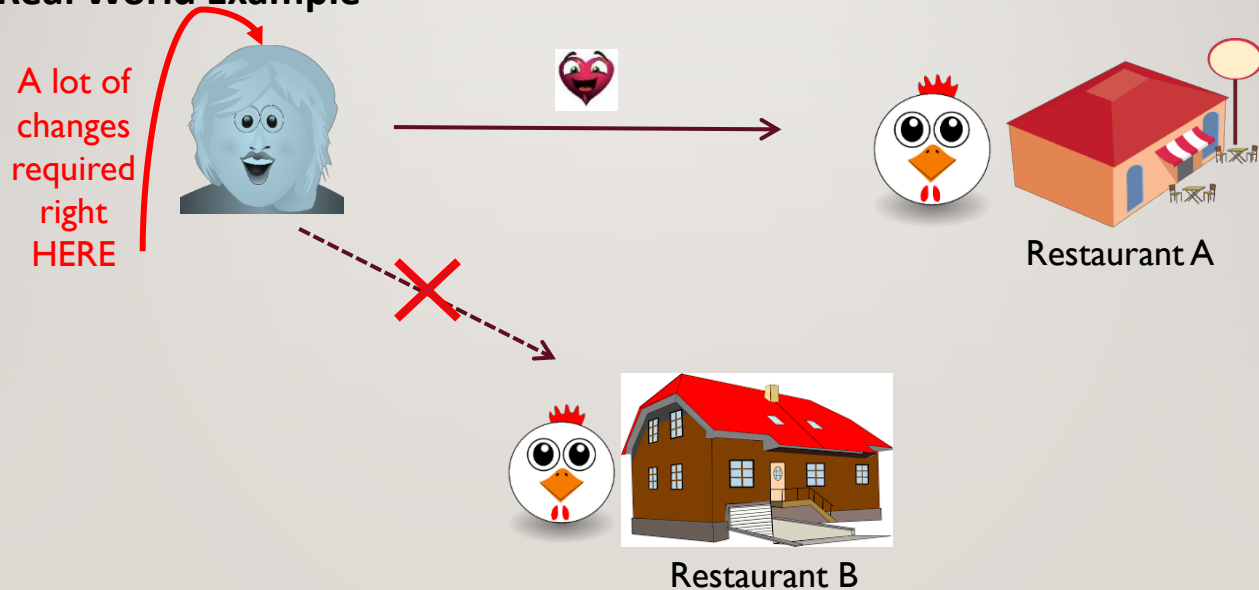
OPEN CLOSED PRINCIPLE

➤ A Real World Example



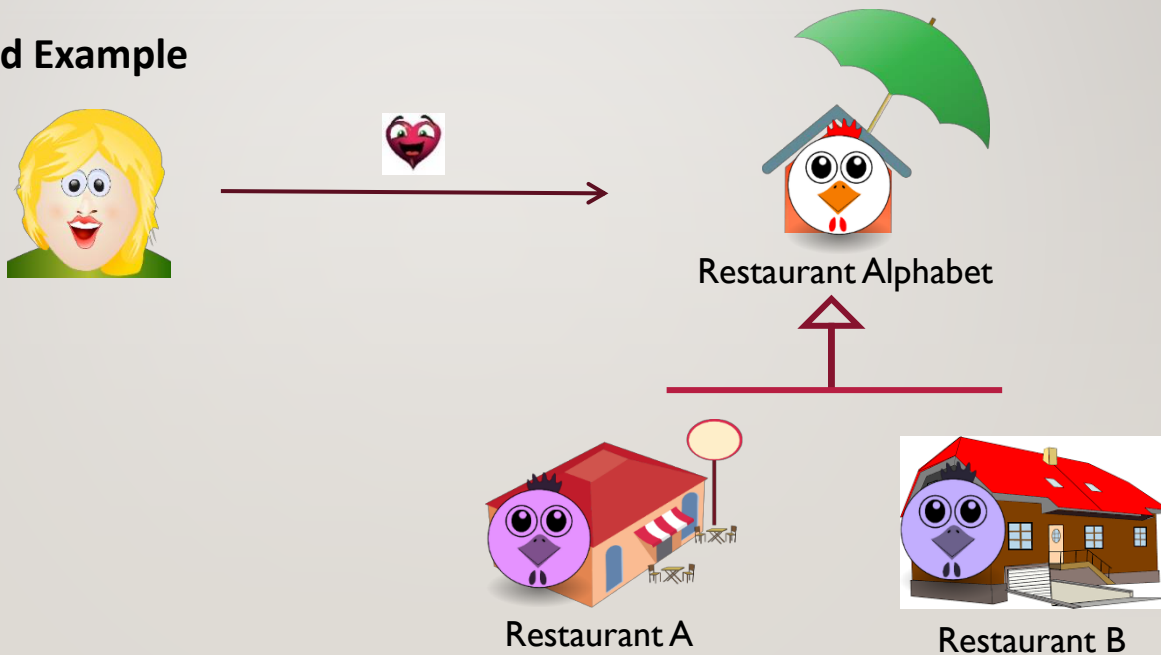
OPEN CLOSED PRINCIPLE

➤ A Real World Example



OPEN CLOSED PRINCIPLE

➤ A Real World Example

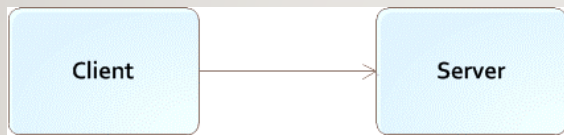


OPEN CLOSED PRINCIPLE

- **A module (or class) should be open for extension, but closed for modifications**

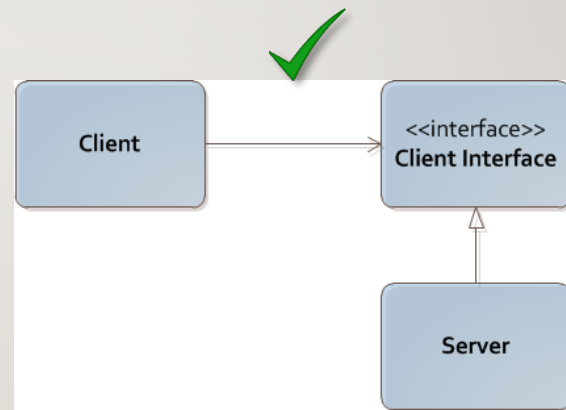
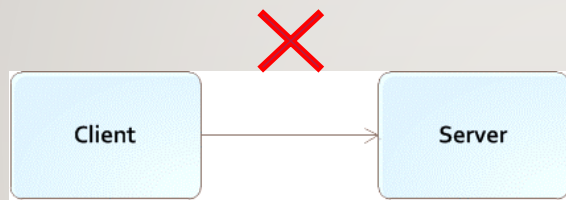
OPEN CLOSED PRINCIPLE

➤ **A module (or class) should be open for extension, but closed for modifications**



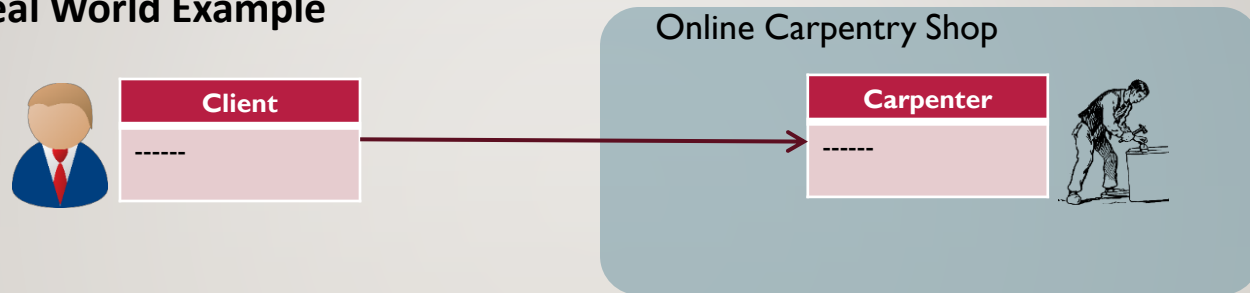
OPEN CLOSED PRINCIPLE

➤ A module (or class) should be open for extension, but closed for modifications



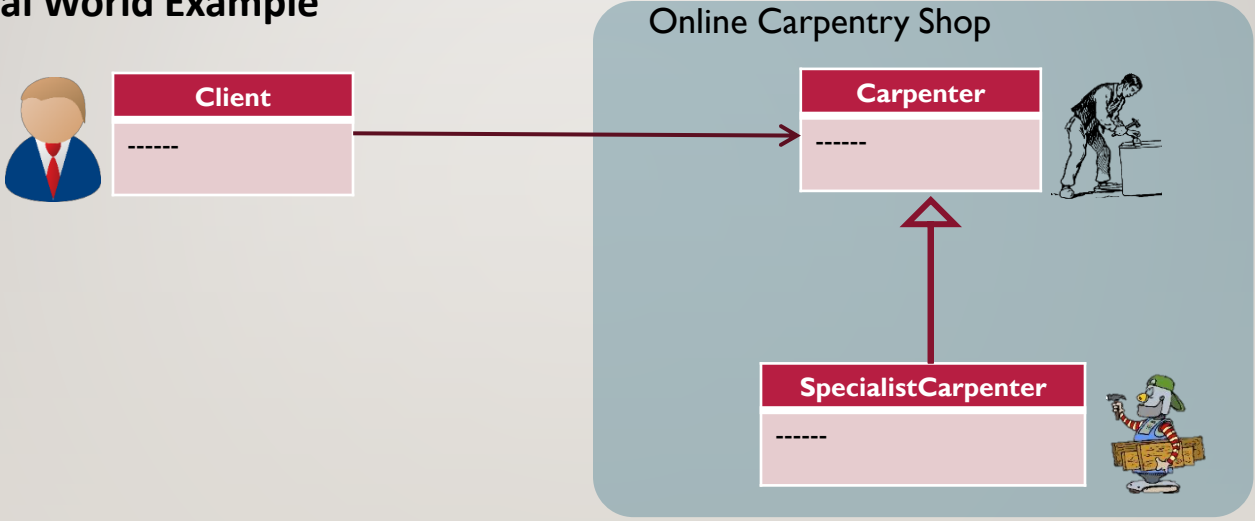
LISKOV SUBSTITUTION PRINCIPLE

➤ A Real World Example



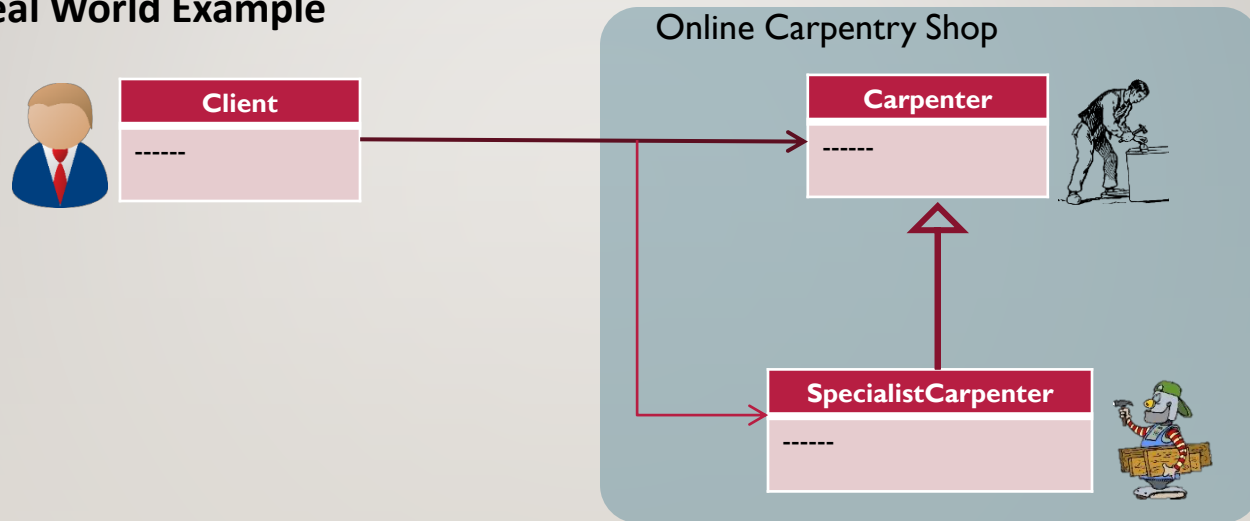
LISKOV SUBSTITUTION PRINCIPLE

➤ A Real World Example



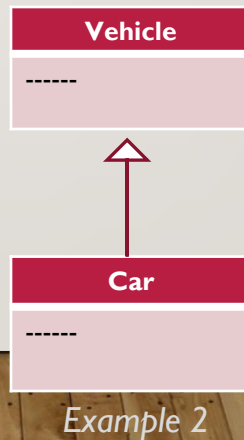
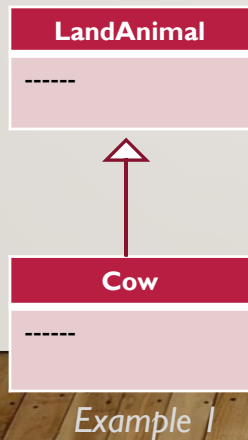
LISKOV SUBSTITUTION PRINCIPLE

➤ A Real World Example



LISKOV SUBSTITUTION PRINCIPLE

- **Derived types must be substitutable for their base types**
- This happens when:
 - Derived class methods **expect no more** and **provide no less** than the base class methods
 - IS-A rule is obeyed



INTERFACE SEGREGATION PRINCIPLE

➤ A Real World Example



INTERFACE SEGREGATION PRINCIPLE

- **Clients should not be forced to depend on methods they do not use**

INTERFACE SEGREGATION PRINCIPLE

- **Clients should not be forced to depend on methods they do not use**

interface SuperOfcMachine

```
print(List<job> job);  
scan(List<job> job);  
photoCopy(List<job> job);  
fax(List<job> job);
```

INTERFACE SEGREGATION PRINCIPLE

- Clients should not be forced to depend on methods they do not use

interface SuperOfcMachine

```
print(List<job> job);  
scan(List<job> job);  
photoCopy(List<job> job);  
fax(List<job> job);
```

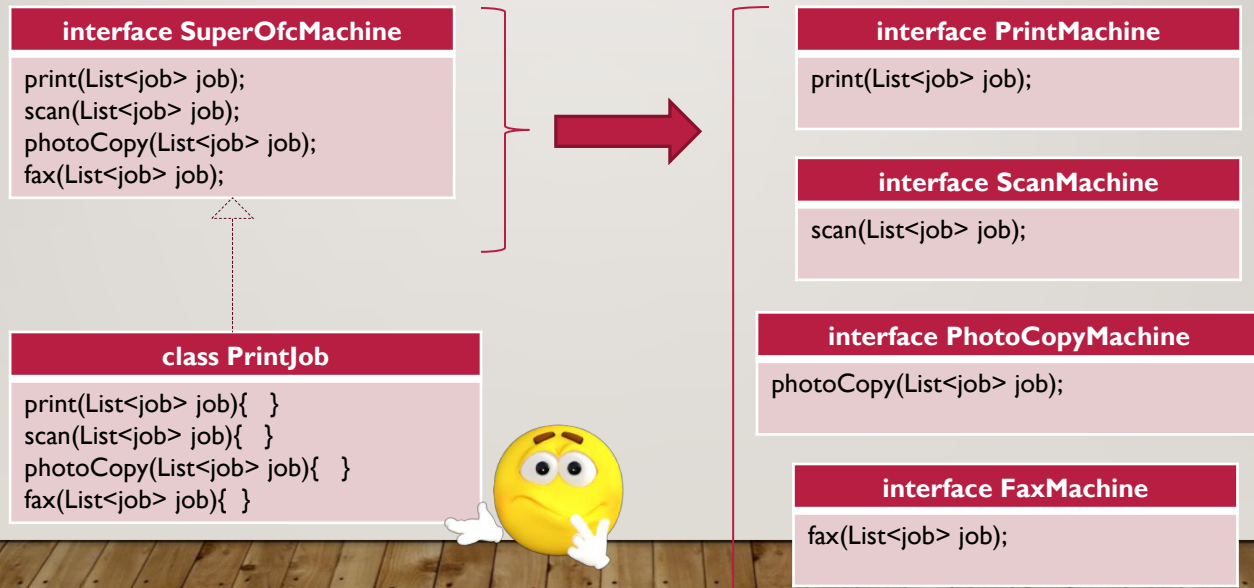
class PrintJob

```
print(List<job> job){ }  
scan(List<job> job){ }  
photoCopy(List<job> job){ }  
fax(List<job> job){ }
```



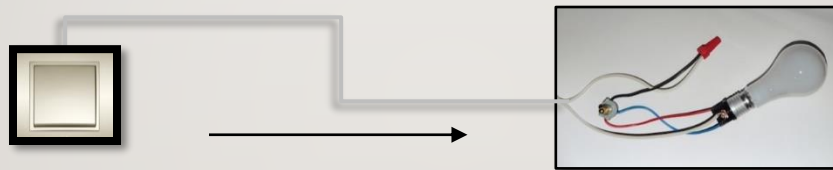
INTERFACE SEGREGATION PRINCIPLE

- Clients should not be forced to depend on methods they do not use



DEPENDENCY INVERSION PRINCIPLE

➤ A Real World Example



DEPENDENCY INVERSION PRINCIPLE

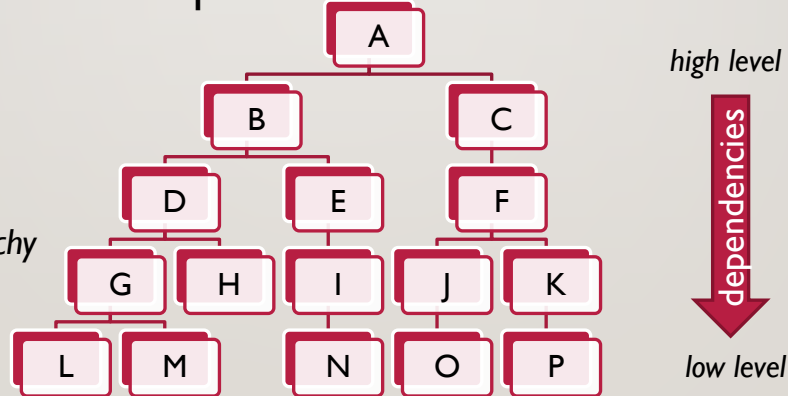
➤ A Real World Example



DEPENDENCY INVERSION PRINCIPLE

- High-level modules should not depend on low-level modules. Both should depend on abstractions
- Abstractions should not depend on details. Details should depend upon abstractions

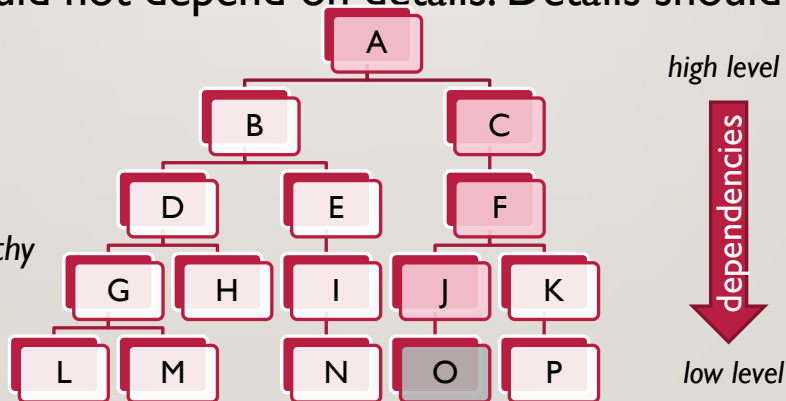
*Example Module
Dependency Hierarchy*



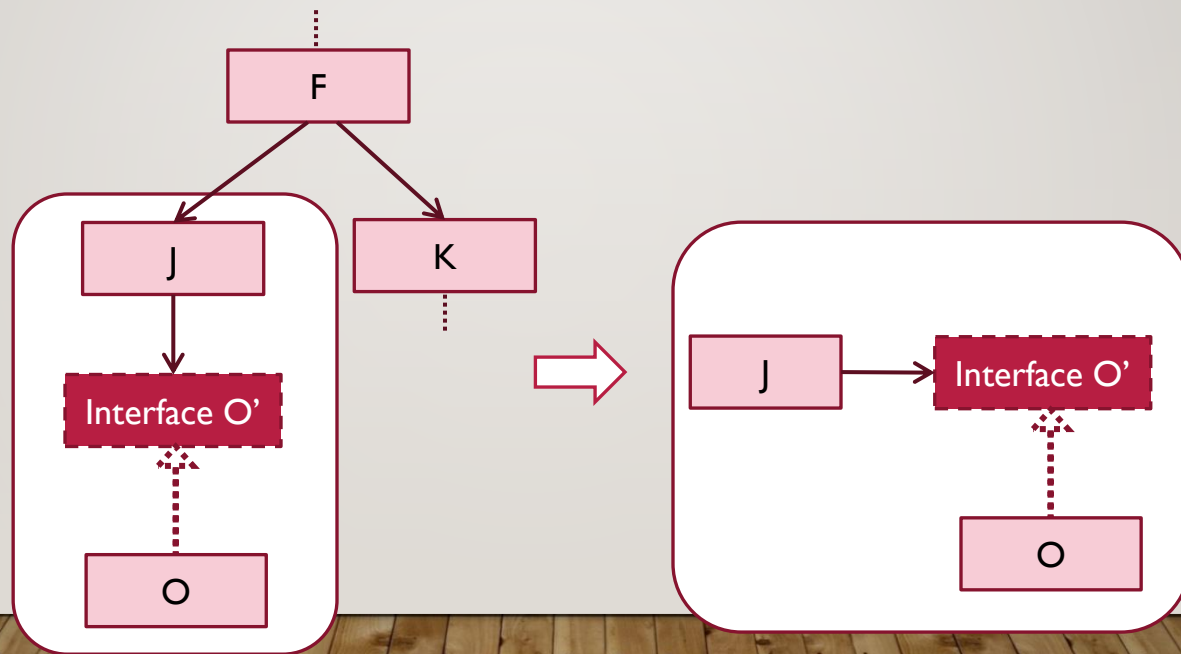
DEPENDENCY INVERSION PRINCIPLE

- High-level modules should not depend on low-level modules. Both should depend on abstractions
- Abstractions should not depend on details. Details should depend upon abstractions

*Example Module
Dependency Hierarchy*



DEPENDENCY INVERSION PRINCIPLE



SOLID REVIEW – NOT EVERYTHING IS “GREEN”!

- **SOLID principles** have often been challenged in practice
- **SRP (ISP)**: Are small classes always easier to understand?
- **DIP (and OCP)**: Should every direct dependency result in at least one additional interface?
- **LSP** does not sound practical enough

SOLID REVIEW – NOT EVERYTHING IS “GREEN”!

- **SOLID principles** have often been challenged in practice
- **SRP (ISP)**: Are small classes always easier to understand?
- **DIP (and OCP)**: Should every direct dependency result in at least one additional interface?
- **LSP** does not sound practical enough

Use SOLID as a ‘guide’ than a ‘goal’?



EXERCISE 3

Which of the following two is a more **robust** object-oriented design?

Justify?

```
public class PlayingCard {
    public static final int SPADE = 101;
    public static final int HEART = 102;
    public static final int CLUB = 103;
    public static final int DIAMOND = 104;

    public static final int ACE = 201;
    public static final int ONE = 202;
    ...
    public static final int KING = 214;

    public int suitValue;
    public int rankValue;
    public boolean faceUp;
}
```

```
List deck =
    new ArrayList( );
```

```
public enum Suit {
    SPADE, HEART, CLUB, DIAMOND };
```

```
public enum Rank {
    ACE, ONE, TWO, THREE, FOUR, FIVE,
    SIX, SEVEN, EIGHT, NINE, TEN,
    JACK, QUEEN, KING };
```

```
public class PlayingCard {

    public final Suit suitValue;
    public final Rank rankValue;
    private boolean faceUp;

    public PlayingCard( Suit suit, Rank rank ) {
        suitValue = suit;
        rankValue = value;
        faceUp = false;
    }

    public void flip( ) {
        faceUp = ! faceUp;
    }
}
```

```
List< PlayingCard > deck =
    new ArrayList< PlayingCard >( );
```

EXERCISE 3 (SOLUTION)

Which of the following two is a more **robust** object-oriented design?

✓ **The right-most implementation**

- **Left** – uses integer constants – the `suitValue` and `rankValue` instance variables can be assigned any (meaningless) *int* value.
 - **Right** – uses enumerated types whose values are constrained and checked at compile time.
- **Left** – state is public and so there is no information hiding. No integrity assurances.
 - **Right** – uses a combination of final and reduced visibility (private). A final instance variable can be assigned only once, and this is handled by the constructor; from then on it can be read only. *faceUp* can be changed only in controlled ways (giving integrity).
- **Right** – uses a constructor to ensure that instances are initialised correctly.
- **Right** – uses a generic type argument for the deck – it can store only **PlayingCard** instances whereas Left's deck can contain any objects.

EXERCISE 4

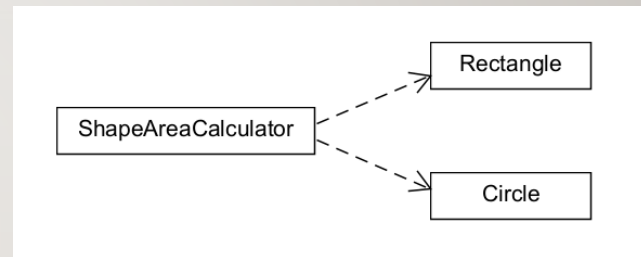
The Java code (download **OCPI.zip** from **Canvas** and unzip into a folder) does not follow open-closed principle.

- **Why?** – write your reason.
- **Refactor** this code such that it follows OCP.

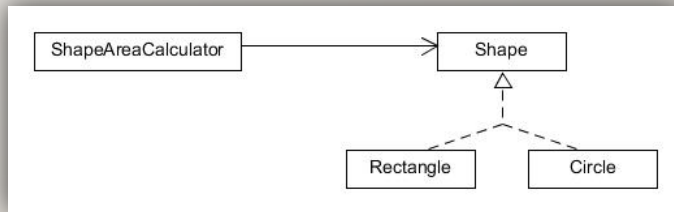
*[Hint] Interpret code in the form of UML class diagram (using any UML tool e.g. **UMLet**, or **by-hand on a paper**)*

EXERCISE 4 (SOLUTION)

- The **ShapeAreaCalculator** class is directly dependent on **Rectangle** and **Circle** classes.
- Adding classes to support more shapes will trigger changes in **ShapeAreaCalculator**
- So, the overall code is open for modification (e.g. in **ShapeAreaCalculator**) and difficult to extend (it is difficult to add classes representing new shapes)
- Refactor this code such that it follows OCP.
 - Create a new **Shape** interface/abstract class and implement **Rectangle** and **Circle** from it
 - Make **ShapeAreaCalculator** dependent on Shape instead of **Rectangle** and **Circle**



Non-OCP compliant



OCP compliant

REVIEW

- The point of software design is to ensure that key objectives (e.g. flexibility, robustness, efficiency etc.) are satisfied
 - Objectives may sometimes conflict
- Undesirable module dependencies cause design smells
- OO design principles are all about managing dependencies
- What's SRP to classes, ISP is to interfaces
- OCP and DIP are quite closely strung up principles
- Using SOLID needs solid CARE!
- Learn by finding instances of the SOLID principles around you

REFERENCES

➤ [Books]

- *Agile Software Development, Principles, Patterns, and Practices* by Robert C Martin
- *Object-Oriented Software Engineering: Practical Software Development using UML and Java* by Timothy Lethbridge and Robert Laganriere

➤ [Media]

- Uncle Bob's Talk at <https://www.youtube.com/watch?v=TMuno5RZNeE>
- SOLID principles <https://raygun.com/blog/solid-design-principles/>

➤ [See more on Canvas Lecture page]

- Week 8 Practice Exercises